COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMIC ENVIRONMENTS

Master thesis

2026 Bc. Filip Sre

COMENIUS UNIVERSITY IN BRATISLAVA
FACULTY OF MATHEMATICS PHYSICS AND INFORMATICS

A MODULAR CLUSTER FOR AI RESEARCH IN RESOURCE-CONSTRAINED ACADEMIC ENVIRONMENTS

Master thesis

Study program:	Applied informatics
Branch of study:	Applied informatics
Department:	Department of Applied Informatics
Supervisor:	Mgr. Ján Kuka, PhD.

Bratislava, 2026 Bc. Filip Sre

47430683

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: Bc. Filip Sršeň
Študijný program: aplikovaná informatika (Jednoodborové štúdium, magisterský II. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: diplomová
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: A Modular Cluster for AI Research in Resource-Constrained Academic Environments
Modulárny klaster pre výskum v AI v akademickom prostredí s obmedzenými zdrojmi

Anotácia: Významná časť výskumu na našej katedre sa opiera o metódy strojového učenia, no dostupná výpočtová infraštruktúra je obmedzená a náročná na správu. Výskumníci pravidelne narážajú na problémy so zdieľaním hardvérových zdrojov, efektívnou prácou s dátovými súbormi a presunom experimentov medzi rôznymi prostrediami. Súčasné postupy – ako manuálne plánovanie, udržiavanie samostatných softvérových inštalácií či kopírovanie dát medzi servermi – komplikujú prácu a znižujú jej produktivitu.

Táto práca sa zameriava na riešenie týchto výziev skúmaním možností návrhu modulárneho klastra prispôsobeného potrebám katedry využitím súčasných technológií na virtualizáciu, kontajnerizáciu a správu klastrov implementovaných ako slobodný softvér/softvér s otvoreným zdrojovým kódom (Proxmox, Docker, Kubernetes, SLURM a pod.). Zámerom je navrhnúť riešenie zlepšujúce prístupnosť, škálovateľnosť a efektivitu práce s ohľadom praktické obmedzenia dostupných zdrojov a pracovné postupy používateľov. Implementácia má demonštrovať, ako tieto môžu uvedené technológie podporovať akademický výskum poskytovaním základu, ktorý uľahčuje spoluprácu, zjednodušuje pracovné postupy a zvyšuje reprodukovateľnosť experimentov strojového učenia.

Cieľ: Hlavným cieľom tejto práce je navrhnúť a implementovať modulárny výpočtový klaster prispôsobený technickým a prevádzkovým obmedzeniam akademického výskumného prostredia. Okrem zabezpečenia zdieľanej výpočtovej kapacity bude klaster slúžiť ako platforma integrujúca do jedného celku plánovanie výpočtových úloh, správu prostredí a sledovanie experimentov. Implementácia systému ukáže, ako možno moderné orchestračné technológie prispôsobiť menšej akademickej infraštruktúre s cieľom zlepšiť reprodukovateľnosť a spoľahlivosť, zjednodušiť údržbu a efektívne využívať dostupné výpočtové zdroje.

Literatúra:
Yoo, A.B., Jette, M.A., Grondona, M (2003). SLURM: Simple Linux Utility for Resource Management. In: JSSPP 2003. Springer.
Domingus, J., Arundel, J. (2022): Cloud Native DevOps with Kubernetes, 2nd Edition. O'Reilly.

47430683

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

Salatino, M. (2024): Platform Engineering on Kubernetes. Manning.
Spišaková, V., Klusáček, D., Hejtmánek, L. (2023). Using Kubernetes in Academic Environment: Problems and Approaches. In: JSSPP 2022. LNCS 13592. Springer
Tim Wickberg (2025). Slinky: Slurm In Kubernetes, Performant AI And HPC Workload Management. In: KubeCon Europe.
Petrosyan, D.A. (2025). Scheduling ML and HPC Jobs with Shoc Platform over Kubernetes. Phys. Part. Nuclei 56, 1581–1585.
Chazapis, A., Nikolaidis, F., Marazakis, M., Bilas, A. (2023). Running Kubernetes Workloads on HPC. In: ISC High Performance 2023. LNCS 13999. Springer.
Misale, C. et al. (2022). Towards Standard Kubernetes Scheduling Interfaces for Converged Computing. In: SMC 2021. CCIS 1512. Springer.

Vedúci: Mgr. Ján Kľuka, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 31.10.2025

Dátum schválenia: 11.11.2025

prof. RNDr. Roman Ďurikovič, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

I hereby declare that I have written this thesis by myself, only with help of referenced literature, under the careful supervision of my thesis advisor.

. . . . . .

Bratislava, 2026 Bc. Filip Sre

Acknowledgement

TODO

Abstract

TODO

Keywords: space debris, machine learning, space object classification

Abstrakt

TODO

Kúové slová: vesmírny odpad, strojové uenie, klasifikácia vesmírnych objektov

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

The widening gap between industry-scale AI compute and the heterogeneous, resource-constrained environments found in academia.

1.1.1 Computing at our department

On a global scale, Comenius University is a rather small institution, with our faculty being an even smaller one, let alone our department. The Department of Applied Informatics has around TODO faculty members and around TODO students. Many of these students are actively doing research in machine learning and adjacent fields, thus requiring computational resources that often go beyond their own personal computers.

The department has its own small server cabinet with a handful of computers equipped with consumer-grade hardware. On request, these machines are available to any student who needs them for their work.

1.1.2 Current state

Currently there are four machines used for compute. Users are provided access to each machine the department manages based on a request from their supervisors. They are also added to a Teams team, which is used to post updates about the machines. It is also used for "reserving" compute slots — users post to the team when, for how long, and on which machine they are going to be computing. Users then use SSH to connect to the selected nodes, move their data to the node, and run their computations.

However, because the computers are standalone machines with no collective management or shared file system beyond user logins, this setup comes with a number of challenges: environment management, data management, and scheduling. These are discussed in detail in the problem statement.

1.2 Problem statement

The complexity of managing distributed AI workloads and the difficulty of providing a "single-pane-of-glass" interface for researchers without heavy DevOps overhead.

1.2.1 Data and storage management

Machine learning often requires large datasets that need to be present for training and evaluating models. Users transfer their datasets onto the machines manually ahead of computation. The datasets then reside on the machines indefinitely. Two problems stem from this:

- When the machine a user has all their data on is being used by someone else, the user needs to either wait for the other job to finish or move the data elsewhere, causing further strain on the machine.
- Users are often not mindful about deleting their unused data. This causes the file system to fill up and requires periodical clean-ups by the administrator.

1.2.2 Environment management

Users manage their environments, packages, and libraries through Conda. As with data, however, these environments are not shared across the machines. This means that whenever users want to switch machines they need to take their Conda environments with them. The result is that the same packages are installed multiple times across the machines, increasing file system utilization further.

1.2.3 Scheduling

Whenever users want to use the machines for compute they are supposed to post into a Teams channel. The post should contain information about their job — mainly the length of the computation and the machine(s) they are selecting. Many times, however, the time estimate is inaccurate, or even worse — users do not post about their job in the channel and just start using the machines. This results in users needing to check each machine individually to confirm whether it is available. There is no centralized monitoring solution, so users manually connect to each node and check its utilization.

1.3 Thesis Contributions

This work has three main goals: research, design, and implementation.

1.3.1 Research

The research goal is to conduct an analysis of compute resource usage at our department. This includes describing different use-cases users have for the system in place and creating typical usage scenarios for both users and administrators. User interviews were conducted with students experienced with the current solution, covering their workflows, data, experiment outputs, hardware and software needs, limiting factors, and pain points.

1.3.2 Design

The design goal is to design solutions that support the identified use-cases. The solution should:

- be able to exist within the existing infrastructure,
- be easily scalable,
- provide additional services,
- support modern AI/ML practices.

1.3.3 Implementation

After providing solutions that address users' needs, the goal is to select a solution and implement it. Specifically, this thesis contributes:

- A modular, Kubernetes-native platform for AI research.
- An intelligent scheduling layer for resource-efficient batch processing.
- A distributed executor for scaling research workloads.

1.4 Thesis Structure

Overview of the subsequent chapters.

Chapter 2

Background & Related Work

2.1 Cloud-Native Orchestration

Fundamentals of Kubernetes and container-based workloads.

2.2 Traditional HPC Scheduling vs. Cloud-Native Orchestration

A central design decision for any compute cluster is the choice of workload management paradigm. The two dominant approaches in research computing are traditional HPC schedulers — exemplified by Slurm — and cloud-native container orchestration platforms — exemplified by Kubernetes. This section compares the two paradigms at a general level, highlighting their respective strengths and limitations in the context of AI/ML research workloads.

2.2.1 Traditional HPC: Slurm

Slurm is the de facto standard workload manager in academic and institutional HPC environments. It provides a mature, battle-tested scheduling engine designed for high-throughput batch processing on large clusters.

Strengths

- **Fair-share scheduling.** Slurm provides sophisticated priority-based scheduling with fair-share policies, allowing administrators to ensure equitable resource distribution across users and groups over time.
- **Efficient resource allocation.** Slurm's job-level resource model (CPU cores, memory, GPUs as discrete allocatable units) is well-suited for tightly-coupled

parallel workloads such as MPI jobs, simulations, and large-scale batch processing.

- **Mature ecosystem.** Decades of development have produced a rich toolchain: advanced reservation systems, array jobs, job dependencies, detailed accounting, and extensive administrative tooling.

- **Proven at scale.** Slurm reliably manages clusters ranging from tens to hundreds of thousands of nodes in production environments worldwide.

Limitations

- **Environment rigidity.** Slurm assumes a shared file system and a uniform software environment across nodes. Users typically rely on environment modules or manual Conda setups, which are not portable across clusters and can lead to version conflicts.

- **Limited support for interactive workloads.** While Slurm can allocate interactive sessions, its design is fundamentally batch-oriented. Long-running interactive services such as Jupyter notebooks, experiment tracking dashboards, or web-based development environments do not fit naturally into the job scheduling model.

- **No native container orchestration.** Slurm can launch containers (e.g., via Singularity/Apptainer), but it does not manage container lifecycles, networking, or service discovery. Containerized services must be managed externally.

- **Monolithic scheduling model.** Slurm's scheduler treats jobs as opaque resource requests. It has no awareness of application-level semantics such as distributed training topology, gang-scheduling requirements, or service dependencies.

2.2.2 Cloud-Native: Kubernetes

Kubernetes is a container orchestration platform originally developed for microservice workloads, but increasingly adopted for AI/ML and batch processing through extensions such as Volcano and Kubeflow.

Strengths

- **Container-native environment management.** Every workload runs in an isolated, reproducible container image. Dependencies, libraries, and runtime ver-

sions are bundled with the application, eliminating environment conflicts and enabling portability.

- **Service-oriented architecture.** Kubernetes natively supports long-running services (JupyterHub, MLflow, Weights & Biases, dashboards) alongside batch workloads, providing a unified platform for both interactive and non-interactive use.
- **Declarative infrastructure.** Cluster state is defined declaratively through manifests and managed via GitOps workflows, enabling reproducible deployments, version-controlled configuration, and easy cluster reconstruction.
- **Rich ecosystem for ML.** Specialized frameworks such as Kubeflow, Volcano (batch scheduling), Kube-Ray (distributed compute), and MLflow integration provide purpose-built tooling for AI/ML workflows including distributed training, hyperparameter tuning, and experiment tracking.
- **Extensibility.** Kubernetes' operator pattern and custom resource definitions (CRDs) allow the platform to be extended with domain-specific scheduling semantics without modifying the core system.

Limitations

- **Operational complexity.** Kubernetes has a steep learning curve and significant operational overhead. Cluster management requires expertise in networking, storage, security, and container registries.
- **Resource overhead.** The control plane (API server, etcd, scheduler, controller manager) consumes resources that may be significant on small clusters, leaving less headroom for compute workloads.
- **Weaker fair-share scheduling by default.** Kubernetes' default scheduler lacks the sophisticated fair-share and priority-decay policies that Slurm provides out of the box. These must be added through extensions such as Volcano.
- **Less mature for tightly-coupled parallel jobs.** Kubernetes is not traditionally designed for MPI-style tightly-coupled parallelism. While support exists (e.g., through MPI operators), it is less mature than Slurm's native MPI integration.

2.2.3 Comparison summary

Neither paradigm is universally superior — the choice depends on workload characteristics and operational requirements. Slurm excels in environments dominated by

traditional batch HPC workloads with homogeneous software stacks. Kubernetes excels where workloads are diverse (batch + interactive + services), environments need to be reproducible and portable, and the research community benefits from modern ML tooling.

For AI/ML research environments specifically, the container-native model offers compelling advantages: reproducible environments, native support for interactive development tools, and a growing ecosystem of ML-specific orchestration frameworks. These benefits must be weighed against the operational complexity and resource overhead that Kubernetes introduces, particularly in resource-constrained academic settings.

2.3 Batch vs. Interactive Computing

The challenge of scheduling high-throughput AI jobs alongside interactive researcher notebooks.

2.3.1 Common use-cases

Through user interviews with students experienced with the current solution, several common use cases were identified:

1. **Script-based batch execution.** Accessing the servers via SSH, copying scripts and datasets onto the machine(s), and running the scripts directly.

2. **Third-party experiment management.** Accessing the servers via SSH, running third-party daemons on the servers, and managing experiments from a web interface. This allows users to run distributed scripts, optimize hyper-parameters of the models, and provides useful metrics about the training process.

3. **Remote development environment.** Using the servers as remote development environments. Users SSH onto the servers and work on their research remotely. This centralization is important for them as they can use thin clients, and if they need to run any quick GPU-heavy calculations to validate their work, there is no need to move to another machine.

2.4 Distributed Machine Learning

Paradigms of data and model parallelism and the need for specialized orchestrators.

2.5 State of the Art

Review of existing frameworks (e.g., Kubeflow) and identifying the gap (e.g., complexity, lack of modularity for specialized academic clusters).

2.5.1 Technologies in academic practice

There is a common trend of using Python for the majority of development in AI research. Many users work with common machine learning frameworks such as TensorFlow or PyTorch.

Some users develop their scripts as Jupyter notebooks. Some convert them into pure Python scripts and run them on the machines. Those who use servers as remote development machines run the notebooks directly.

Users also require metrics from their experiments, whether produced by the ML frameworks themselves or by an ML platform such as Weights & Biases.

2.5.2 Hardware requirements

Computation

Almost all users use the machines to train machine learning models. This usually requires GPUs to accelerate the process. Users also want to utilize multiple GPUs to further speed up the process.

Some users — namely those that work with tabular data — want to run ETL pipelines on the data. This is normally a CPU- and memory-heavy task.

Data storage

A significant portion of research is done in computer vision. This use-case is the most resource-intensive, as image or video datasets can reach hundreds of gigabytes depending on the project.

More standard machine learning projects require far less data storage — in the range of megabytes to several gigabytes.

There are also users who work with many small files, which can be quite IOps-intensive.

Chapter 3

System Architecture Design

3.1 Introduction

3.1.1 Design Objectives

Modularity, Scalability, and Resource Efficiency.

3.1.2 Design Philosophy

Cloud-native, microservice-based approach using Kubernetes.

3.2 High-Level System Model

3.2.1 Layered Abstraction Framework

Identity, Interface, Orchestration, and Lifecycle.

3.2.2 System Interaction Model

End-to-end workflow from auth to distributed training.

3.2.3 The Modular Interconnectivity

Decoupling via standardized interfaces (APIs/K8s CRDs).

3.3 Detailed Component Architecture

3.3.1 Identity and Governance Layer

Keycloak integration for multi-tenancy.

3.3.2 Research Interface and Environment Provisioning

JupyterHub and Harbor (private registry).

3.3.3 Distributed Orchestration and Scheduling Engine

Volcano

Batch scheduling and gang-scheduling for AI workloads.

Kube-Ray

Distributed compute scaling.

3.3.4 Experiment Lifecycle and Metadata Management

MLflow integration for tracking metrics/artifacts.

3.4 Design for Resource Constraint Mitigation

3.4.1 Advanced Scheduling Strategies

Volcano queueing and priority mechanisms.

3.4.2 Elasticity and Resource Elasticity

K8s/Ray dynamic scaling.

3.5 Summary

Concluding synthesis of modular design meets the problem statement.

Chapter 4

Implementation & Technical Realization

4.1 Deployment Strategy

Using GitOps and Helm-based manifests to manage the cluster lifecycle.

4.2 Implementing the Orchestration Engine

4.2.1 Configuring Volcano for Academic Constraints

Priority queues and gang-scheduling.

4.2.2 Integrating Kube-Ray for Distributed Scaling

Distributed compute scaling.

4.3 Building the Research Interface

How the API Gateway and JupyterHub provide the unified interface for the end-user.

4.4 Data and Lifecycle Automation

The integration of Harbor for image stability and MLflow for scientific experiment tracking.